

Функциональная верификация цифровой аппаратуры

Лабораторная работа. Тестовое окружение для системной верификации

Фролов П.В,

April 15, 2026

Организация тестового стенда для системной верификации

Раскладка тестовой программы по памяти

- 0 .. – тестовая программа;
- 0x100000000 .. – программа начального старта (инженерный бут).

В тестовом окружении микропроцессоров Эльбрус бинарным образам ПНС и тестовой программы соответствуют разные файлы.

Маршрут получения бинарного образа тестовой программы для помещения в тестовый стенд

Два образа: тестовой программы и программы начального старта.

- 1 Препроцессирование (`cpp: .s → .cppout`);
- 2 трансляция (`las: .cppout → .elf`);
- 3 привязка к пространству памяти тестового стенда (`ld: .elf → .elf`);
- 4 извлечение бинарного кода из `.elf`-контейнера (`objcopy: .elf → .bin`);
- 5 преобразование бинарного кода в текстовый формат для чтения тестовым стендом (`bin2vbin: .bin → __test__.o`).

- Внутренняя документация:
`http://www.lab.sun.mcst.ru/honey/elbrus\_core\_iset/iset/current/iset`
- Общедоступный документ: `https://www.mcst.ru/doc/elbrus\_prog/html/index.html`

Перечень задач для реализации

- 1 Сортировка массива слиянием.
- 2 Быстрая сортировка массива (сортировка Хоара).
- 3 Быстрый поиск элемента в отсортированном массиве.
- 4 Перемножение матриц.
- 5 Топологическая сортировка графа.
- 6 Пирамидальная сортировка массива.
- 7 Быстрое дискретное преобразование Фурье.

Вариативность по типам сортируемых данных.

Лабораторная работа по системной верификации (1а)

Окружение ВК

Необходимо разработать

- 1 функцию `f` на языке ассемблера Эльбрус (v6 или v7);
- 2 тестовую программу для запуска функции `f` на симуляторе вычислительного комплекса.

Требования

- Исходный код должен компилироваться ассемблером `las` после препроцессирования стандартным препроцессором языка `C`.
- Результат компиляции должен запускаться на симуляторе ВК с помощью скрипта `r3`.

Лабораторная работа по системной верификации (16)

Рекомендации и указания

- Необходимые функции f вспомогательные ресурсы (память) следует резервировать в коде тестовой программы статически и передавать указатели на них в дополнительных аргументах.
- Во избежание дополнительной настройки программных стеков следует избегать использования рекурсии.
- Содержание вызывающей функцию f тестовой программы:
 - базовая настройка системы;
 - входные данные для целевой функции в виде статически включенных в тестовую программу данных;
 - эталонный результат исполнения функции (для встроенной в тест самопроверки);
 - подготовка аргументов, вызов функции f и проверка результата её исполнения.
- Ассемблерный файл с исходным кодом функции f достаточно включить в файл с обрамляющим кодом.
- Программные соглашения вызова процедур:
https://www.mcst.ru/doc/elbrus_prog/html/chapter9.html#id34

Лабораторная работа по системной верификации (2)

Окружение ОС (уровень приложения)

Задание

- 1 Разработать функцию `main` (C или C++), которая:
 - формирует исходные данные для функции `f`;
 - вызывает `f`;
 - проверяет результат (например, с помощью стандартных функций языковой библиотеки (STL)).
- 2 Собрать исполняемый файл для запуска на функциональной модели уровня приложения.
- 3 Осуществить то же на потактовой функциональной модели уровня приложения.
- 4 Реализовать эталонную версию функции `f` на используемом языке программирования высокого уровня.
- 5 Сравнить быстродействие "эталонной" версии и реализованной на ассемблере.

Ключи к решению задач

- Необходимые функции f вспомогательные ресурсы (память) следует резервировать перед вызовом и передавать указатели на них в дополнительных аргументах.
- Во избежание дополнительной настройки программных стеков рекомендуется реализация целевой функции без использования рекурсии.
- Для запуска целевой функции на модели машины необходим обрамляющий код:
 - базовая настройка системы;
 - входные данные для целевой функции в виде статически включенных в тестовую программу данных;
 - данные для проверки результатов исполнения;
 - подготовка аргументов, вызов и проверка результата исполнения.
- Ассемблерный файл с исходным кодом целевой функции достаточно включить в файл с обрамляющим кодом.

Функциональные модели МП архитектуры Эльбрус уровня приложений

Архитектурные

- `/auto/vgr/simul/last/e32c_sim`
- `/auto/vgr/simul/last/e2c3_sim`

Потактовой точности

- `/auto/vgr/simul/last/e32c_csim`
- `/auto/vgr/simul/last/e2c3_csim`

Кросс-компиляторы

- `/home/cf/e2k/mcst.rel-29-0/bin/lxx_i`
- `/home/cf/e2k/mcst.rel-29-0/bin/lcc_i /auto/vgr/bin/las64_i`

Пример

```
//! \file    mini.s
/// \author opium
/// \date    2017.12.08
/// \brief   Тест исполнения инженерного бута.
///
/// $Id: mini.s 22479 2021-12-23 15:50:40Z opium $
```

```
#include "init.s"
```

```
.global user_text_start
.text
user_text_start:
    TEST_PASSED
```

Пример

```
/auto/vgr/bin/r3 --el=8v7 -vc /auto/vgr/users/opium/elbrus-tests/mini.s
```

```
Running cpp for test (cpp -D__iset__=7 -D__elbrus_8v7__ -I/auto/vgr/users/opium/elbrus-tests/
```

```
↪ -I/auto/vgr/users/opium/elbrus-tests/./LOC_INCL -I/auto/hard/e8v7/e8v7.last/dbg/GLOB_INCL
```

```
↪ /auto/vgr/users/opium/elbrus-tests/mini.s -o mini.s.cppout)... ok
```

```
Running las for test (/auto/vgr/bin/las64_i --no_strict_delay --emulate-asel -mcpu=elbrus-v7 mini.s.cppout -o
```

```
↪ mini.s.o)... ok
```

```
Running ld for test
```

```
↪ (/auto/vgr/bin/elbrus-comp/binutils.cross/mcst.rel-i-1.binutils/2021-09-23/e2k-v2-linux.cross/bin/e2k-linux-ld
```

```
↪ -Tmini.sbP_by.lds -Aelbrus-v7 mini.s.o -o mini.s.elf)... ok
```

```
Running ldis for test (/auto/vgr/bin/ldis64_i --emulate-asel -mcpu=elbrus-v7 mini.s.o)... ok
```

```
Running objcopy for test
```

```
↪ (/auto/vgr/bin/elbrus-comp/binutils.cross/mcst.rel-i-1.binutils/2021-09-23/e2k-v2-linux.cross/bin/e2k-linux-objcopy
```

```
↪ -O binary -R .note -R.line -R.nodis.text -R .comment -S mini.s.elf mini.s.bin)... ok
```

```
Running objdump for test
```

```
↪ (/auto/vgr/bin/elbrus-comp/binutils.cross/mcst.rel-i-1.binutils/2021-09-23/e2k-v2-linux.cross/bin/e2k-linux-objdump
```

```
↪ -D mini.s.elf -melbrus-v7)... ok
```

```
Running bin2vbin for test (/auto/vgr/bin/bin2vbin mini.s.bin mini.s.vbin)... ok
```

Запуск на программной модели вычислительного комплекса

Ключи `-ms` указывают `r3` запустить скомпилированные тесты на программной модели вычислительного комплекса:

```
/auto/vgr/bin/r3 --el=8v7 -vms  
    --fsim-opts='-vei'  
    /auto/vgr/users/opium/elbrus-tests/mini.s
```

В отладочной выдаче `r3` (включаемой ключом `-v`) помимо прочего видно исполняемую команду запуска симулятора:

```
/auto/vgr/simul/last/e8v7_ms -mc -cf3 -D -cw -cn -cis4 -cb -cj \  
-rsuppress_dbg_data:false -rmegahertz:2500 -vei \  
-rROM_IMAGE:__bios__.o -rROM_LOADER:true __test__.o
```

Справка по ключам запуска программной модели вычислительного комплекса

```
/auto/vgr/simul/last/e8v7_ms --help
```

Дизассемблер

```
<0000000005c8> HS      xxxxxxxx :  
                SS      xxxxxxxx  ct %ctpr3  
                        ipd 3
```

..

! function 'user_text_start', entry = 12, value = 0x000e20, size = 0x008, sect = ELF_TEXT num =

```
<000000000e20> HS      xxxxxxxx user_text_start:  
                CS1     xxxxxxxx  wait fl_c_mode = 0, macp = 0, mem_mod = 0, int = 0, mt = 0  
                        , lal = 0, las = 0, sal = 0, sas = 0, trap = 0, ma_c =  
                        , ld_c = 0, st_c = 0, all_e = 1, all_c = 1
```

Трасса исполнения

*** #0.0 0x5c8/0x5c8.114 *****

HS 0 xxxxxxxx :CT BRANCH_CT %ctpr3 (0xe20) taken always

SS 0 xxxxxxxx ipd(0x3) rp_opc(0) eap(0) bap(0) vfdi(0) abn(0) abp(0) alc(0) aa(0)

TEST_OK

*** #0.0 0xe20/0xe20.115 *****

HS 0 xxxxxxxx :

CS1 0 xxxxxxxx WAIT mem_mod(0) int(0) mt(0) lal(0) las(0) sal(0) sas(0)

trap(0) ma_c(0x1) fl_c(0x1) ld_c(0) st_c(0) all_e(0x1) all_c(0x1)